

Avance 04 — Revisión e Integración de Módulos

Proyecto: Modelado MP con coherencia de caché MESI — Producto punto paralelo

Curso: CE-4302 Arquitectura de Computadores II

Grupo: Kevin Rodríguez Lanuza, Saúl Gómez Ramírez, Daniel Cob Beirute

Entrega (avance 4): integración y pruebas funcionales básicas

Fecha del informe: (poner la fecha en Google Docs si quieren otra distinta)

1. Objetivo del avance

Validar la **inicialización** e **integración** de los módulos principales del modelo MP y comprobar su interacción mínima mediante casos de prueba:

- Memoria principal (mem.c)
- Cachés privadas por PE (cache.c)
- Bus de interconexión y handlers (bus.c / handlers.c)
- Processing Elements (PEs) y su hilo de ejecución (pe.c)

Se registran logs detallados para permitir trazabilidad y depuración del protocolo MESI.

2. Resumen de lo probado y formato de logs

Toda la ejecución incluye printf de depuración para cada evento relevante:

- Inicializaciones: [MAIN] Memoria inicializada., [MAIN] Caché X inicializada..., [BUS] Initialized.
- Eventos del bus: [BUS] Señal <id> recibida de PEX (addr=Y) y [BUS] Ejecutando handler ...

- Transiciones MESI: [CACHE] Transición de estado: <old> -> <new>
- Hits / Misses: [PEX] HIT en set ... / [PEX] MISS en set ...
- Lecturas / Escrituras: [PEX] Leyó ... desde MEM[...], [PEX] Escribió ... en MEM[...]

Estos logs permiten reconstruir la trazabilidad de cada operación.

3. Casos de prueba (CT — identificador, pasos, resultado esperado y observado)

Nota: cada caso incluye un identificador (ID) para control de pruebas y rastreo.

CT-READ-01 — Lectura inicial (single read)

ID: CT-READ-01

Objetivo: validar miss en primera lectura y que bus lea de memoria.

Precondición: memoria inicializada (valores por defecto = 0.0).

Pasos:

1. Lanzar mp_mesi con PEs ejecutando cache_read(pe->cache, addr=0, pe_id).

Resultado esperado:

- PE0 genera MISS y envía BusRd.
- Bus lee memoria y la caché de PE0 transiciona Invalid -> Shared.
- PE0 imprime el valor leído (por defecto 0.0).

Resultado observado (logs):

- [PE0] MISS en set 0, pidiendo BusRd
- [BUS] Línea no encontrada → leyendo de memoria
- [CACHE] Transición de estado: Invalid -> Shared
- [PE0] Leyó 0.000000 desde MEM[0]

Estado: OK — comportamiento conforme esperado.

CT-READ-02 — Lecturas concurrentes múltiples (Shared behavior)

ID: CT-READ-02

Objetivo: validar que sólo la primera lectura provoca BusRd; lecturas posteriores sean HIT (o “hit indirecto” según el diseño).

Precondición: memoria con valor conocido o resultado CT-READ-01 ejecutado antes.

Pasos:

1. Ejecutar PEs 0..3 leyendo la misma dirección 0 de forma concurrente.

Resultado esperado:

- Solo PE0 (el que llega primero) hace MISS y BusRd.
- PE1..PE3 hacen HIT o al menos leen el valor sin provocar BusRd extra.

Resultado observado (logs previos):

- En ejecuciones previas, cuando las lecturas arrancan casi simultáneas: se vieron varios MISS porque las cachés locales todavía estaban en Invalid al comienzo y los hilos entraron prácticamente al mismo tiempo.
- Después de ajustar el flujo y sincronización, la ejecución correcta reporta MISS solo para la primera lectura y HIT en las demás (cuando la temporización lo permite).

Estado: Parcial — comportamiento sensible a timing de hilos; la corrección en bus/handlers y en cache_read reduce misses innecesarios. Para validar de forma determinista, ejecutar lecturas secuenciales (ver CT-SEQR-01).

CT-SEQR-01 — Lecturas secuenciales deterministas

ID: CT-SEQR-01

Objetivo: validar comportamientos MESI en orden determinista.

Pasos:

1. cache_read(&caches[0], 0, 0)
2. cache_read(&caches[1], 0, 1)
3. cache_read(&caches[2], 0, 2)

Resultado esperado:

- PE0: MISS -> bus -> memoria; PE0 Invalid -> Shared.

- PE1: HIT (detecta que linea existe en otra caché y lee valor) -> no BusRd.
- PE2: HIT.
Resultado observado: Con las correcciones aplicadas (bus devolviendo datos desde caches y cache_read respetando S/E), el resultado es conforme.
Estado: OK.

CT-WRITE-01 — Escritura en dirección única (write miss + BusRdX)

ID: CT-WRITE-01

Objetivo: validar WRITE miss → BusRdX → Invalidación en otras caches y Invalid -> Modified en la local.

Pasos:

1. cache_write(&caches[0], 0, 3.14, 0)
2. cache_read(&caches[0], 0, 0)

Resultado esperado:

- PE0: WRITE miss -> BusRdX -> Invalid -> Modified.
- Cuando lee inmediatamente: HIT en M -> puede downgrade si otra lectura externa aparece.

Resultado observado (logs):

- [PE0] WRITE miss en set 0, enviando BusRdX
- [BUS] Señal 1 recibida de PE0 (addr=0)
- [CACHE] Transición de estado: Invalid -> Modified
- [PE0] Escribió 3.140000 en MEM[0] (si se realizó write-back según política; en nuestro modelo write-back se escribe a memoria cuando se flush o reemplaza)
- [PE0] HIT en set 0 (way 0)
- [PE0] Leyó 3.140000 desde MEM[0]
Estado: OK.

CT-WRITE-CONC-01 — Escrituras concurrentes a la misma dirección

ID: CT-WRITE-CONC-01

Objetivo: validar invalidaciones (Shared -> Invalid) y que el último escritor quede Modified.

Pasos:

1. Lanzar PEs 0..3 que escriben valores diferentes en addr=0 prácticamente simultáneo.

Resultado esperado:

- Cada BusRdX provoca invalidación en otras cachés.
- Solo el último escritor mantiene Modified al final; los demás terminan con Invalid.

Resultado observado (logs):

- Las trazas muestran varios BusRdX y las correspondientes invalidaciones. Al final, la caché del último PE que escribió está en Modified.

Estado: OK.

4. Cambios / correcciones realizadas en el código (resumen técnico)

A continuación se listan las correcciones aplicadas con comentarios para el informe:

1. `cache.c`

- Implementada política básica de reemplazo: `find_line_to_replace(set)` — ahora no siempre se usa `lines[0]`.
- `cache_read`:
 - Revisión correcta de hits (considera `valid && tag == tag`) y estados M/S/E para devolver dato sin hacer BusRd.
 - En MISS se ejecuta `bus_broadcast(BUS_RD, ...)` y se lee `mem_read(addr)` por defecto; luego se comprueba si otras caches tienen la línea (M/S/E) para obtener el dato real y hacer downgrade M→S si aplica.
 - Mutex para proteger sección crítica: evita races entre hilos.

- Logs de debug conservados y detallados ([CACHE], [PE#]).
- cache_write:
 - Si hit S/E → M y se notifica al bus con BUS_UPGR para invalidar otras caches si aplica.
 - Si miss → BUS_RDX, se reemplaza la línea en una way libre y se marca Modified.
 - Evitar write-back inmediato en cada write (modelo write-back): write-back ocurre cuando otra caché tiene M y es forzada a escribir a memoria por el handler del bus.
- Se dejó trazabilidad completa: hits, misses, transiciones MESI.

2. bus.c / handlers.c

- bus_broadcast con mutex global del bus para serializar handlers (evitar races en handlers).
- Handlers (BusRd, BusRdX, BusUpgr, BusWB) actualizados para:
 - BusRd: revisar otras caches, si hay M escribir a memoria (mem_write) y hacer downgrade M->S; si E, degradar a S.
 - BusRdX: invalidar S/E en otras caches; si hay M, escribir a memoria antes de invalidar.
 - BusUpgr: invalidar líneas S en otras cachés (cuando local pasa S->M por upgrade).
 - BusWB: forzar write-back desde el PE solicitante si tiene M.
- Todos los printf de bus y handlers preservados para debug.

3. memory.c

- mem_init, mem_read, mem_write con mutex para proteger acceso concurrente.
- Inicialización con 0.0 por defecto (se puede cargar dataset para el producto punto).

4. Sincronización y robustez

- Uso de `pthread_mutex_t` en bus y caches para evitar condiciones de carrera.
- Evitar funciones no definidas (`bus_broadcast_read`) — todo hecho con `bus_broadcast` + comprobación directa de otras caches.

5. Resultados obtenidos y evidencia (resumen)

Se presentan los resultados resumidos a partir de las ejecuciones realizadas (logs completos guardados en la carpeta logs/):

- **Inicialización:** Memoria y caches inicializadas correctamente, bus registrado con punteros a caches.
- **Lecturas simples:** PE0 hace MISS → BusRd → memoria; PE0 transiciona I->S. Lecturas posteriores se vuelven HIT si la temporización lo permite.
- **Escrituras simples:** WRITE miss en dirección única → BusRdX → Invalid->Modified en caché local. Lectura inmediata en la misma PE produce HIT.
- **Escrituras concurrentes:** Se observaron múltiples BusRdX y invalidaciones correctas; el último escritor queda con Modified.
- **Transiciones MESI:** Invalid->Shared, Invalid->Modified, Modified->Shared (cuando otro PE lee y hay write-back) y Shared->Invalid (cuando hay BusRdX de otro PE) todas registradas en logs.

6. Qué se arregló en este avance (lista concreta)

1. Evitar reemplazo estático (siempre `lines[0]`) — ahora se usa `find_line_to_replace`.
2. `cache_read` corregida para no enviar BusRd si hay hit en S/E/M local.
3. `cache_read` obtiene valores correctos si otra caché tenía M/S/E (downgrade y write-back si procede).
4. `cache_write` maneja S/E→M y notifica BUS_UPGR / BUS_RDX.

5. Handlers del bus actualizados para respetar MESI y realizar write-back cuando hay M.
6. Añadidos mutex para bus y caches (sincronización por hilos).
7. Conservados y ampliados printf de debug para trazabilidad completa.

7. Métricas registradas y cómo obtenerlas

El modelo está preparado para registrar (se pueden extender contadores en cada cache/PE):

- cache_misses por PE (incrementar en cache_read y cache_write cuando ocurre miss).
- invalidations_count por PE (incrementar en handlers cuando invalidan una línea).
- read_ops / write_ops por PE (incrementar en cada operación).
- transitions_log (lista de strings o contador por transición MESI).
- mem_access_log (lecturas/escrituras a memoria).

Cómo añadir/extraer métricas:

- En cache.c declarar campos en Cache (ej: uint64_t misses, reads, writes, invalidations;) y hacer printf al final del run o escribir a csv.
- Sugerencia: agregar una función report_stats() en main.c que recorra caches[] y muestre métricas por PE al finalizar.

8. Problemas pendientes / limitaciones detectadas

1. **Timing / non-determinismo:** cuando las lecturas son *concurrentes* y entran exactamente al mismo tiempo, se pueden observar múltiples MISS porque cada caché aún no ve el resultado de la otra. Esto es inherente a la simultaneidad; para pruebas deterministas usar secuencias o barriers.

2. **Instrumentación automática:** no se generaron todavía CSV con métricas por corrida; lo dejo como tarea inmediata (ver plan).
3. **Interfaz de visualización (stepping, breakpoints):** no implementada como GUI; se sugiere añadir un modo stepping con espera por tecla o un archivo de control.
4. **Cobertura de ISA / ejecución del producto punto completo:** Actualmente solo se prueban accesos a caché / bus. Falta integrar el ensamblador/loader del ISA para ejecutar el caso producto punto completo (cargar vectores A, B, ejecutar FMUL/FADD/LOAD/STORE, reducir sumas).